STEP in practice: AP203, AP214 and AP242

Authors: Marsette Vona martyvona@gmail.com and Claude, 2026

Contents

1 Introduction 2
2 The Part 21 file format 3
2.1 Structure 3
2.2 Attribute values 3
2.3 Simple and complex instances 3
2.4 Attribute order 4
3 EXPRESS in five minutes 5
4 Units and representation contexts 5
5 Representations, geometry and topology 6
5.1 Representations 6
5.2 Geometry 6
5.3 Topology (B-rep) 6
6 Products and their shapes 7
6.1 Linked representations 7
7 Assemblies 7
7.1 The structure 7
7.2 Placing a component: the implicit method 8
7.3 Placing a component: the explicit method (mapped items) 8
7.4 How real files deviate 9
7.5 External references 9
8 Colours, styles and layers 9
9 Validation properties 10
10 Tessellated geometry (AP242) 11
11 What real files look like 11
12 Complete examples 12
12.1 A single part: a cylinder with a red top face 12
12.2 An assembly: a pin through a hole in a disc 14
12.3 The same assembly, placed by mapped items (the explicit method) 17
13 References 18

1 Introduction

This is a practical introduction to the STEP files that CAD systems exchange: what is in them, how the pieces fit together, and how real files depart from the standard. It assumes you know CAD basics (B-rep solids, NURBS, assemblies) but nothing about STEP. Everything here can be checked against the references listed at the end.

STEP is ISO 10303, a family of standards for exchanging product data. Three kinds of parts matter for reading files:

- **The file format**, ISO 10303-21 ("Part 21"): a plain-text list of numbered records. Most `.stp`/`.step` files are Part 21. (There is also an XML form, `.stpx`, not covered here.)
- **The modelling language**, ISO 10303-11, EXPRESS: a schema language, a bit like a class hierarchy with typed attributes. Every Part 21 file is an instance population of one EXPRESS schema.
- **Application protocols (APs)**: the schemas themselves, each built from shared "integrated resources" (geometry and topology in Part 42, product structure in Part 41, representations in Part 43, presentation in Part 46, and so on).

The three APs that carry almost all mechanical CAD data:

AP	Schema name in files	Typical content
AP203 edition 1	`CONFIG_CONTROL_DESIGN`	Solids and assemblies, with configuration management (approvals, dates, people). No colours in the schema itself, though writers add them anyway.
AP203 edition 2	`AP203_CONFIGURATION_CONTROLLED_3D_DESIGN_OF_MECHANICAL_PARTS_AND_ASSEMBLIES_MIM_LF`	AP203 updated with colours, layers and more.
AP214	`AUTOMOTIVE_DESIGN`	The automotive AP: solids, assemblies, colours, layers, validation properties, kinematics. Very widely used by CAD exporters.
AP242	`AP242_MANAGED_MODEL_BASED_3D_ENGINEERING_MIM_LF`	The merger and successor of AP203 and AP214, adding tessellated geometry, semantic PMI (tolerances, datums) and more. Several editions.

For geometry, topology, assemblies, units and colours the three are nearly the same. The entities and their attributes mostly coincide, and a reader written for one handles the others with small adjustments. AP242 is close to a superset of AP214 for this data; the two differ mostly in kinematics.

The header names the schema, sometimes with an object identifier giving the edition:

```
FILE_SCHEMA(('AUTOMOTIVE_DESIGN { 1 0 10303 214 3 1 1 }'));
FILE_SCHEMA(('AP242_MANAGED_MODEL_BASED_3D_ENGINEERING_MIM_LF { 1 0 10303 442 3 1 4 }'));
```

An AP's EXPRESS comes in two forms. The **short form** is a small schema that mostly imports what it needs from the integrated resources and modules (`USE FROM` and `REFERENCE FROM` clauses). The **long form** is the same schema with every import expanded into one self-contained schema. Files use the same entities either way, but the long form is what a reader wants, since it needs no other schemas. The `_MIM_LF` in the names stands for "MIM long form".

An AP also has two levels of model:

- **ARM** (Application Reference Model): the user-level concepts ("part", "assembly component usage"), written for domain experts. Files never contain ARM entities.
- **MIM** (Module Interpreted Model; AIM, Application Interpreted Model, in older APs): what files actually contain. The AP documents how each ARM concept maps onto MIM entities, which is why, for example, a "part" becomes the `PRODUCT` to `PRODUCT_DEFINITION` chain of section 6.

2 The Part 21 file format

2.1 Structure

```
ISO-10303-21;
HEADER;
FILE_DESCRIPTION(('a bracket'),'2;1');
FILE_NAME('bracket.stp','2024-03-08T15:00:31',('author'),('org'),
  'preprocessor version','originating system','authorization');
FILE_SCHEMA(('AUTOMOTIVE_DESIGN { 1 0 10303 214 3 1 1 }'));
ENDSEC;
DATA;
#10=CARTESIAN_POINT('',(0.,0.,0.));
#11=DIRECTION('',(0.,0.,1.));
#12=DIRECTION('',(1.,0.,0.));
#13=AXIS2_PLACEMENT_3D('',#10,#11,#12);
ENDSEC;
END-ISO-10303-21;
```

- The **header** has three mandatory records. `FILE_NAME`'s fifth and sixth fields (preprocessor version and originating system) identify the writer, which matters because writers have their own quirks.
- The **data section** is a list of instances: `#id = ENTITY_NAME(attribute, attribute, ...);`. Ids are positive integers, unique, in any order, and needn't be dense. References (`#13`) may point forwards or backwards.
- Whitespace and line breaks are insignificant; comments are `/* ... */`.
- Keywords are uppercase in practice; readers should accept any case.

2.2 Attribute values

Written	Meaning
`12`, `-3`	integer
`1.5`, `0.`, `1.E-07`	real: always has a decimal point (`5` is an integer, `5.` a real)
`'text'`	string, with `''` for a quote
`.T.`, `.F.`, `.U.`, `.METRE.`	enumeration (booleans and logicals are enumerations too)
`#12`	reference to another instance
`"0FF"`	binary: hex digits, the first giving the number of unused bits
`$`	unset (an absent optional attribute)
`*`	derived: the attribute's value is computed by the schema, not stored
`(a, b, c)`	list, set or bag (aggregates may nest)
`LENGTH_MEASURE(5.)`	typed parameter: a value tagged with its type, needed where the schema allows several types (a SELECT)

Strings are ASCII in the file. Other characters use escapes: `\X2\00C4\X0\` for UTF-16 code units (here U+00C4, capital A with diaeresis), `\X4\0001F7CA\X0\` for code points beyond 16 bits, and the older `\X\C4` (one ISO 8859-1 byte) and `\S\`/`\P\` code-page escapes. A literal backslash is `\\`.

2.3 Simple and complex instances

Most instances are **simple**: one entity type, with the attributes of that entity and all its supertypes. Some instances belong to several entity types at once, where EXPRESS allows types to combine (`ANDOR`, `AND`). These are written as **complex instances**, also called external mapping:

```
#20=(LENGTH_UNIT()NAMED_UNIT(*)SI_UNIT(.MILLI.,.METRE.));
```

Rules for complex instances:

- Each partial record, `TYPE(...)`, holds **only the attributes that type itself declares**, not inherited ones. Here `NAMED_UNIT` declares `dimensions` and `SI_UNIT` declares `prefix` and `name`. `LENGTH_UNIT` declares nothing, so its partial is empty.
- Partials are supposed to be in alphabetical order; a few writers don't comply, and nothing depends on the order.
- `*` appears where a subtype redeclares an inherited attribute as derived. `SI_UNIT` derives `NAMED_UNIT.dimensions` from its name, so the file writes `*`.

Units and representation contexts (sections 4 and 5) are nearly always complex instances, and so are rational B-splines.

Example: the same kind of curve, simple and complex. `B_SPLINE_CURVE_WITH_KNOTS` inherits from `B_SPLINE_CURVE`, which inherits (through `BOUNDED_CURVE`, `CURVE` and `GEOMETRIC_REPRESENTATION_ITEM`) from `REPRESENTATION_ITEM`. Of those, only three entities declare explicit attributes:

Entity	Its own attributes
`REPRESENTATION_ITEM`	`name`
`B_SPLINE_CURVE`	`degree`, `control_points_list`, `curve_form`, `closed_curve`, `self_intersect`
`B_SPLINE_CURVE_WITH_KNOTS`	`knot_multiplicities`, `knots`, `knot_spec`

A non-rational cubic spline is a **simple instance**: one entity name, with all inherited attributes first, in that order:

```
#1=CARTESIAN_POINT('',(0.,0.,0.));
#2=CARTESIAN_POINT('',(1.,2.,0.));
#3=CARTESIAN_POINT('',(3.,2.,0.));
#4=CARTESIAN_POINT('',(4.,0.,0.));
#5=B_SPLINE_CURVE_WITH_KNOTS('',3,(#1,#2,#3,#4),.UNSPECIFIED.,.F.,.F.,(4,4),(0.,1.),.UNSPECIFIED.);
```

In order: the name `''`; degree 3, four control points, curve form, not closed, not self-intersecting; then knot multiplicities (4, 4) on knots 0 and 1 (a Bezier segment), and the knot type.

A rational spline must also be a `RATIONAL_B_SPLINE_CURVE`, which adds `weights_data`. No single entity combines the knot vector with the weights, so the instance belongs to several entity types at once and is written as a **complex instance**. Every entity in the set is listed, its supertypes included, each holding only its own attributes. This quarter circle of radius 1 is a quadratic with weights 1, $\sqrt{2}/2$, 1:

```
#11=CARTESIAN_POINT('',(1.,0.,0.));
#12=CARTESIAN_POINT('',(1.,1.,0.));
#13=CARTESIAN_POINT('',(0.,1.,0.));
#14=(BOUNDED_CURVE()
     B_SPLINE_CURVE(2,(#11,#12,#13),.CIRCULAR_ARC.,.F.,.F.)
     B_SPLINE_CURVE_WITH_KNOTS((3,3),(0.,1.),.UNSPECIFIED.)
     CURVE()
     GEOMETRIC_REPRESENTATION_ITEM()
     RATIONAL_B_SPLINE_CURVE((1.,0.707106781186548,1.))
     REPRESENTATION_ITEM(''));
```

`BOUNDED_CURVE()`, `CURVE()` and `GEOMETRIC_REPRESENTATION_ITEM()` are empty because those entities declare no attributes, but they must still appear: the partials list the instance's whole set of types. The `name` that a simple instance puts first now sits in the `REPRESENTATION_ITEM` partial at the end, because of the alphabetical order. Rational B-spline surfaces follow the same pattern (`B_SPLINE_SURFACE`, `B_SPLINE_SURFACE_WITH_KNOTS`, `RATIONAL_B_SPLINE_SURFACE` with a grid of weights).

2.4 Attribute order

For a simple instance, attributes appear in EXPRESS declaration order: the supertypes' attributes first (depth first, in `SUBTYPE OF` order, each supertype once), then the entity's own. For example `CARTESIAN_POINT` inherits `name` from `REPRESENTATION_ITEM`, so `CARTESIAN_POINT('name',(x,y,z))` is (`name`, `coordinates`). Reading attributes by name

needs the schema; reading them by position doesn't, which is why tolerant readers keep the positional data even when the schema doesn't match.

3 EXPRESS in five minutes

```
ENTITY axis2_placement_3d
  SUBTYPE OF (placement);
  axis          : OPTIONAL direction;
  ref_direction : OPTIONAL direction;
DERIVE
  p : LIST [3:3] OF direction := build_axes(axis, ref_direction);
END_ENTITY;
```

- **Entities** have explicit attributes (stored in the file), derived attributes (computed; not in the file unless a subtype redeclares an inherited attribute as derived, which produces `*`), and inverse attributes (back-references; never in the file).
- **Defined types** (`TYPE length_measure = REAL;`), **enumerations** and **SELECT types** (a union of entities or types). A value in a SELECT that isn't an entity reference must be written as a typed parameter, `LENGTH_MEASURE(5.)`, so the reader knows which member it is.
- **WHERE rules** and global rules constrain populations. Readers rarely check them, and files often violate them.

The EXPRESS schemas of the APs and integrated resources are freely available (section 13). They are the reference for every attribute layout in this document.

4 Units and representation contexts

Every geometric representation names a **context** that fixes its units and precision:

```
#30=(LENGTH_UNIT()NAMED_UNIT(*)SI_UNIT(.MILLI.,.METRE.));
#31=(NAMED_UNIT(*)PLANE_ANGLE_UNIT()SI_UNIT($,.RADIAN.));
#32=(NAMED_UNIT(*)SI_UNIT($,.STERADIAN.)SOLID_ANGLE_UNIT());
#33=UNCERTAINTY_MEASURE_WITH_UNIT(LENGTH_MEASURE(1.E-07),#30,'distance_accuracy_value','');
#34=(GEOMETRIC_REPRESENTATION_CONTEXT(3)GLOBAL_UNCERTAINTY_ASSIGNED_CONTEXT((#33))
     GLOBAL_UNIT_ASSIGNED_CONTEXT((#30,#31,#32))REPRESENTATION_CONTEXT('',''));
```

- **SI units** (`SI_UNIT(prefix, name)`): names include `METRE`, `SQUARE_METRE`, `CUBIC_METRE`, `RADIAN`, `STERADIAN`, `GRAM` (the base for mass, so a kilogram is `.KILO.`/`.GRAM.`), `SECOND`. A prefix on `SQUARE_METRE` or `CUBIC_METRE` is conventionally raised to that power (millimetre squared).
- **Conversion-based units** define a unit as a multiple of another:

  ```
  #40=LENGTH_MEASURE_WITH_UNIT(LENGTH_MEASURE(25.4),#30);
  #41=(CONVERSION_BASED_UNIT('INCH',#40)LENGTH_UNIT()NAMED_UNIT(#42));
  #42=DIMENSIONAL_EXPONENTS(1.,0.,0.,0.,0.,0.,0.);
  ```

  Degrees are the usual plane angle unit: `CONVERSION_BASED_UNIT('DEGREE', PLANE_ANGLE_MEASURE_WITH_UNIT(PLANE_ANGLE_MEASURE(0.0174532925199433), radian))`. The factor's precision varies between writers.
- **Derived units** combine others with exponents (`DERIVED_UNIT((DERIVED_UNIT_ELEMENT(inch, 3.)))` is a cubic inch) and are used mainly for areas and volumes in validation properties.
- **Uncertainty** is the modelling tolerance: points closer than this are the same point.

Real files use millimetres most often, then inches, centimetres, metres and feet. Each representation has its own context, and an assembly may mix units: an inch part placed in a millimetre assembly is normal, and the placement must convert. A few writers declare an area or volume unit as a plain length unit (`VOLUME_UNIT` with `CONVERSION_BASED_UNIT('INCH', ...)`); what they mean is its cube.

5 Representations, geometry and topology

5.1 Representations

A **representation** is a set of items in a context: `REPRESENTATION(name, items, context_of_items)`. For shape, the common subtypes are:

Entity	Holds
`SHAPE_REPRESENTATION`	anything; often just a placement, linked to the real geometry (section 6)
`ADVANCED_BREP_SHAPE_REPRESENTATION`	B-rep solids (`MANIFOLD_SOLID_BREP`, `BREP_WITH_VOIDS`) with full topology and exact geometry
`MANIFOLD_SURFACE_SHAPE_REPRESENTATION`	open or closed shells without a solid (`SHELL_BASED_SURFACE_MODEL`)
`FACETED_BREP_SHAPE_REPRESENTATION`	polyhedral solids
`GEOMETRICALLY_BOUNDED_SURFACE_SHAPE_REPRESENTATION`	surfaces without topology (trimmed surfaces in a `GEOMETRIC_SET`)
`GEOMETRICALLY_BOUNDED_WIREFRAME_SHAPE_REPRESENTATION`, `EDGE_BASED_WIREFRAME_SHAPE_REPRESENTATION`	curves only
`TESSELLATED_SHAPE_REPRESENTATION` (AP242)	triangle meshes (section 10)

A representation's items usually include an `AXIS2_PLACEMENT_3D` at the origin as well as the solids. Assembly placements refer to it (section 7).

5.2 Geometry

- `CARTESIAN_POINT('', (x, y, z))`; `DIRECTION('', (dx, dy, dz))`, not necessarily normalized; `VECTOR('', direction, magnitude)`.
- **`AXIS2_PLACEMENT_3D('', location, axis, ref_direction)`**, a right-handed frame. Its z axis is `axis` (default (0, 0, 1)). Its x axis is `ref_direction` projected perpendicular to z (default (1, 0, 0), or (0, 1, 0) if z is along x). Its y axis is $z \times x$. (A reader that computes $x \times z$ instead gets a left-handed frame. Relative transforms between two such frames still come out right, so the mistake shows only when one placement is checked on its own.) As a 4×4 matrix, the columns are x, y, z and the location. Directions in files are often slightly off unit length or perpendicularity, so normalize and orthogonalize.
- Curves: `LINE`, `CIRCLE`, `ELLIPSE` (each positioned by a placement), `B_SPLINE_CURVE_WITH_KNOTS` (degree, control points, form, closed, self-intersect, knot multiplicities, knots, knot type), rational versions as complex instances, `TRIMMED_CURVE`, `COMPOSITE_CURVE`, `POLYLINE`.
- Surfaces: `PLANE`, `CYLINDRICAL_SURFACE`, `CONICAL_SURFACE`, `SPHERICAL_SURFACE`, `TOROIDAL_SURFACE` (each with a placement), `B_SPLINE_SURFACE_WITH_KNOTS`, `SURFACE_OF_REVOLUTION`, `SURFACE_OF_LINEAR_EXTRUSION`, `OFFSET_SURFACE`.

5.3 Topology (B-rep)

```
#100=MANIFOLD_SOLID_BREP('',#101);
#101=CLOSED_SHELL('',(#110,#111,...));            faces of the solid
#110=ADVANCED_FACE('',(#120),#130,.T.);           bounds, surface, same_sense
#120=FACE_OUTER_BOUND('',#121,.T.);               a loop and its orientation
#121=EDGE_LOOP('',(#122,#123,#124,#125));
#122=ORIENTED_EDGE('',*,*,#140,.T.);              an edge used in a direction
#140=EDGE_CURVE('',#150,#151,#160,.T.);           start vertex, end vertex, curve, same_sense
#150=VERTEX_POINT('',#170);
```

```
#170=CARTESIAN_POINT('',(0.,0.,0.));
#160=LINE('',#170,#180);
#130=PLANE('',#13);
```

- A face is bounded by loops (one outer, the rest holes) and lies on a surface. `same_sense` says whether the face normal agrees with the surface normal.
- Edges are shared between faces, with each face using an edge through an `ORIENTED_EDGE`. The `*`s in `ORIENTED_EDGE` are its derived start and end vertices.
- `BREP_WITH_VOIDS` adds inner shells. `OPEN_SHELL` occurs in surface models.

6 Products and their shapes

The product structure ties a part's identity to its geometry. A minimal single part:

```
#1=APPLICATION_CONTEXT('automotive design');
#2=PRODUCT_CONTEXT('',#1,'mechanical');
#3=PRODUCT_DEFINITION_CONTEXT('part definition',#1,'design');
#4=PRODUCT('PN-1001','Bracket','',(#2));                    id (part number), name, description, contexts
#5=PRODUCT_DEFINITION_FORMATION('A','',#4);                 a version of the product
#6=PRODUCT_DEFINITION('design','',#5,#3);                   a view of that version (the "part definition")
#7=PRODUCT_DEFINITION_SHAPE('','',#6);                      the definition's shape, as a property
#8=SHAPE_DEFINITION_REPRESENTATION(#7,#9);                  ... represented by this representation
#9=ADVANCED_BREP_SHAPE_REPRESENTATION('',(#13,#100),#34);   placement and solid, in context #34
```

- **`PRODUCT`** is the part or assembly as an identity. Its `id` is usually the part number and its `name` the human name; either may be empty, and writers fill them inconsistently.
- **`PRODUCT_DEFINITION`** (PD) is what assemblies and shapes attach to. One product may have several (for different life-cycle stages), but in CAD exports there is almost always one per product.
- AP203 variants add `PRODUCT_DEFINITION_FORMATION_WITH_SPECIFIED_SOURCE` and many configuration-management entities (approvals, persons, dates, security classifications), which carry no geometry.

6.1 Linked representations

Often the definition's own representation holds only a placement, and the geometry is in another representation linked by a `SHAPE_REPRESENTATION_RELATIONSHIP` (SRR) without a transformation:

```
#9=SHAPE_REPRESENTATION('',(#13),#34);
#20=ADVANCED_BREP_SHAPE_REPRESENTATION('',(#13,#100),#34);
#21=SHAPE_REPRESENTATION_RELATIONSHIP('','',#9,#20);
```

The order of `rep_1` and `rep_2` in such links varies, so treat them as undirected. A part may link several geometric representations: several bodies, a B-rep plus a tessellated alternative, or supplemental construction geometry.

7 Assemblies

7.1 The structure

An assembly is a product whose definition is related to its components' definitions by **`NEXT_ASSEMBLY_USAGE_OCCURRENCE`** (NAUO):

```
#50=NEXT_ASSEMBLY_USAGE_OCCURRENCE('1','bolt 1','',#6_asm,#6_bolt,'B1');
```

The attributes are id, name, description, the relating (parent) definition, the related (child) definition and a reference designator. Each NAUO is one use of the child in the parent: a bolt used four times has four NAUOs. Root products are definitions that are nobody's child. Files may have several roots, for example several unconnected parts.

7.2 Placing a component: the implicit method

The usual way to place an occurrence: a `CONTEXT_DEPENDENT_SHAPE_REPRESENTATION` (CDSR) connects the occurrence, through its own `PRODUCT_DEFINITION_SHAPE`, to a representation relationship carrying a transformation:

```
#51=PRODUCT_DEFINITION_SHAPE('','',#50);                        the shape of the occurrence
#52=(REPRESENTATION_RELATIONSHIP('','',#9_bolt,#9_asm)
     REPRESENTATION_RELATIONSHIP_WITH_TRANSFORMATION(#53)
     SHAPE_REPRESENTATION_RELATIONSHIP());
#53=ITEM_DEFINED_TRANSFORMATION('','',#13_bolt,#60);          transform_item_1, transform_item_2
#54=CONTEXT_DEPENDENT_SHAPE_REPRESENTATION(#52,#51);
#60=AXIS2_PLACEMENT_3D('',...);                                 a placement among the assembly's items
```

The conventions (PDM Schema Usage Guide, section 4.4.2):

- `rep_1` is the **component's** representation and `rep_2` the **assembly's**.
- `transform_item_1` is a placement in the component's representation, and `transform_item_2` one in the assembly's.
- The component is placed so that frame `transform_item_1` lands on frame `transform_item_2`. With M_1 and M_2 the frames' matrices, a point moves from component to assembly coordinates by

  $$x_{\text{assembly}} = M_2 \, S \, M_1^{-1} \, x_{\text{component}}$$

  where S scales by the component's length unit expressed in the assembly's (25.4 for an inch component in a millimetre assembly).

`transform_item_1` is usually the component's origin placement, which is the identity, so the transform reduces to M_2. That is why mistakes in the general formula go unnoticed in most files. One widely used implementation computes $M_1^{-1} M_2$ instead, which agrees only when one of the frames is the identity.

A `CARTESIAN_TRANSFORMATION_OPERATOR_3D` may replace the `ITEM_DEFINED_TRANSFORMATION`, but this is rare.

7.3 Placing a component: the explicit method (mapped items)

The PDM Schema Usage Guide (section 4.4.1) gives a second way to place components, which represents the assembled geometry explicitly. It uses **`MAPPED_ITEM`**, STEP's general mechanism for placing a whole representation inside another as if it were one item (a block or instance reference, in CAD terms):

```
#70=REPRESENTATION_MAP(#13_bolt,#9_bolt);           mapping_origin (a placement in the bolt), mapped_representation
#71=MAPPED_ITEM('bolt 1',#70,#72);                  mapping_source, mapping_target
#72=AXIS2_PLACEMENT_3D('',...);                     where the origin goes, in the containing representation
```

The mapped representation's contents appear in the containing representation, moved so that `mapping_origin` lands on `mapping_target`: $M(\text{target})\, S\, M(\text{origin})^{-1}$. One `REPRESENTATION_MAP` can serve many `MAPPED_ITEM`s, which is how geometry is reused.

For an assembly:

- **The assembly's representation contains one mapped item per component**, so its contents are the placed components.
- **The occurrence is linked to its mapped item:** the NAUO's `PRODUCT_DEFINITION_SHAPE` has an ordinary `SHAPE_DEFINITION_REPRESENTATION`, with no CDSR, whose representation holds that occurrence's mapped item. That tells a reader which mapped item places which occurrence.

```
#121=MAPPED_ITEM('pin',#111,#108);                  in the assembly's representation, and ...
#410=NEXT_ASSEMBLY_USAGE_OCCURRENCE('2','pin','',#102,#302,$);
#411=PRODUCT_DEFINITION_SHAPE('','',#410);
#412=SHAPE_DEFINITION_REPRESENTATION(#411,#413);
#413=SHAPE_REPRESENTATION('pin occurrence',(#121),#15);   ... in the occurrence's own representation
```

Section 12 has a complete example. The implicit method is far more common in CAD exports. Readers handle the explicit one less well: OCCT, for example, needs the mapped items listed in the assembly's representation as well as the occurrences', and then loses the components' names.

Mapped items also appear without any assembly:

- **inside single parts**, for repeated features
- **in drawings and PMI**, where their targets are view boxes and camera models rather than placements

There is one trap for readers. A representation that contains mapped items also references, transitively, everything they map. A reader that collects "everything a representation references" as one part's geometry will include the mapped geometry. If it also places the mapped representations separately, that geometry is counted twice.

7.4 How real files deviate

Readers need to handle all of these:

- **`rep_1` and `rep_2` swapped** (the assembly first). This is common: in one large sample, about a quarter of all placements. Find out which representation belongs to the child definition (through its SDR, and SRR links from there), and invert the transform if needed.
- **The transformation's items swapped** relative to the representations (`transform_item_1` lying in `rep_2`). The usual fix, the one OCCT applies, is to swap the items back to match the representations. At least one writer (Alibre) instead means the items in the order written, so swapping them back mis-assembles its files. The two cases look identical structurally; the header's originating system tells them apart.
- **No NAUO at all**: an assembly expressed only as `MAPPED_ITEM`s in one representation, mapping other products' shapes. Treating each such mapped item as an occurrence recovers the structure.
- **A `PRODUCT_DEFINITION_SHAPE` where a NAUO should name a `PRODUCT_DEFINITION`**: follow its `definition`.

7.5 External references

A large assembly may keep its parts in separate files. The component's definition is linked to a document:

```
#80=DOCUMENT_TYPE('geometry');
#81=DOCUMENT_FILE('bolt.stp','','',#80,'',$);
#82=APPLIED_DOCUMENT_REFERENCE(#81,'',(#6_bolt));
#83=PROPERTY_DEFINITION('external definition','',#81);
```

The component's local representation then holds only a placement, and its geometry is in `bolt.stp`, relative to the referencing file. The CAx-IF practice on external references adds more detail, including document representation types and file formats.

8 Colours, styles and layers

Presentation comes from ISO 10303-46. A **`STYLED_ITEM`** applies styles to one representation item (a solid, a shell, a face, a curve) or to a whole representation:

```
#200=COLOUR_RGB('',0.8,0.1,0.1);
#201=FILL_AREA_STYLE_COLOUR('',#200);
#202=FILL_AREA_STYLE('',(#201));
#203=SURFACE_STYLE_FILL_AREA(#202);
#204=SURFACE_SIDE_STYLE('',(#203));
#205=SURFACE_STYLE_USAGE(.BOTH.,#204);
#206=PRESENTATION_STYLE_ASSIGNMENT((#205));
#207=STYLED_ITEM('color',(#206),#100);                        styles the solid #100
#208=MECHANICAL_DESIGN_GEOMETRIC_PRESENTATION_REPRESENTATION('',(#207),#34);
```

- **Colours:**
  - `COLOUR_RGB` has components in [0, 1], interpreted as sRGB. A few writers use 0 to 255; dividing by the largest component (as OCCT does) recovers the colour.

  - `DRAUGHTING_PRE_DEFINED_COLOUR('red')` comes from a fixed list: black, white, red, green, blue, yellow, cyan, magenta.
- **Surface styles:** `SURFACE_STYLE_USAGE(side, ...)` with a fill-area colour. Sometimes `SURFACE_STYLE_RENDERING_WITH_PROPERTIES` is added, carrying a colour and `SURFACE_STYLE_TRANSPARENT(t)`. When both are present, the rendering colour is the intended one.
- **Curve styles:** `CURVE_STYLE(name, font, width, colour)`, for edges and wireframe. A curve style colours curves and edges, never faces: a solid's faces take their colour from surface styles only.
- **Point styles:** `POINT_STYLE` with a marker.
- **Boundary styles:** `SURFACE_STYLE_BOUNDARY(curve_style)` colours only a surface's boundary curves. Some writers (Unigraphics among them) colour their solids *only* this way, and they mean the faces; viewers generally show those faces in that colour.
- **Null style:** `NULL_STYLE(.NULL.)`, written as a typed enumeration among the styles, means "display with the receiver's default".
- **Inheritance:** styles pass down the topology (from solid to shell to face) and down the assembly (from assembly to components). Per the practice, a colour on an assembly applies to everything in it. A surface style on a `GEOMETRIC_SET` or a `CURVE_BOUNDED_SURFACE` colours the surfaces in it.
- **Overrides:** a face styled differently from its solid should use **`OVER_RIDING_STYLED_ITEM(name, styles, item, over_ridden_style)`**. In practice many writers style faces with plain `STYLED_ITEM`s; viewers then usually let the more specific (face) style win.
- **Per-instance styles:** `CONTEXT_DEPENDENT_OVER_RIDING_STYLED_ITEM` styles one instance in an assembly. Its `style_context` is the list of representation relationships (the CDSRs' `REPRESENTATION_RELATIONSHIP_WITH_TRANSFORMATION`s) from the root down to that instance, a chain that can be any depth. The PDM Schema's `SPECIFIED_HIGHER_USAGE_OCCURRENCE` (SHUO) names one instance deep in an assembly (a NAUO as seen through its parents' NAUOs), for properties that apply to that instance only.
- **Per-view styles:** a `PRESENTATION_STYLE_BY_CONTEXT` applies only in its context (a particular view), not to the default appearance.
- **Invisibility:** `INVISIBILITY((styled item or layer, ...))` hides what it lists.
- **Layers:** `PRESENTATION_LAYER_ASSIGNMENT(name, description, (items))` groups representation items (solids, faces, curves).
- **Containers:** the `STYLED_ITEM`s are collected in a `MECHANICAL_DESIGN_GEOMETRIC_PRESENTATION_REPRESENTATION` or a `DRAUGHTING_MODEL`. Readers can find styled items directly; the container matters mainly to writers.

9 Validation properties

CAD systems can record what they compute for the geometry, so the receiver can check its own reading:

```
#300=PROPERTY_DEFINITION('geometric validation property','volume of the bracket',#7);
#301=MEASURE_REPRESENTATION_ITEM('volume measure',VOLUME_MEASURE(12345.6),#310);
#302=REPRESENTATION('volume',(#301),#34);
#303=PROPERTY_DEFINITION_REPRESENTATION(#300,#302);
#304=PROPERTY_DEFINITION('geometric validation property','centroid',#7);
#305=CARTESIAN_POINT('centre point',(10.,20.,5.));
#306=REPRESENTATION('centroid',(#305),#34);
#307=PROPERTY_DEFINITION_REPRESENTATION(#304,#306);
#310=DERIVED_UNIT((#311));
#311=DERIVED_UNIT_ELEMENT(#30,3.);                        cubic millimetres
```

- **Property definitions:** the name is `'geometric validation property'` (some writers use underscores) or `'assembly validation property'`. The definition is a product's `PRODUCT_DEFINITION_SHAPE`, for the whole part, or a `SHAPE_ASPECT`, for one solid, shell or face. A shape aspect's geometry is given by `GEOMETRIC_ITEM_SPECIFIC_USAGE` or by a small shape representation of its own.
- **Item names identify the value:**
  - `'volume measure'`, `'surface area measure'`, `'centre point'`
  - for unconnected geometry: `'independent surface area measure'`, `'curve length measure'`

  - `'bounding box corner point'`, and point and child counts
- **Pick values by these names.** Several representations may share one property definition, and representation names are often empty.
- Points are in the representation's length unit; measures carry their own unit.

Checking these against your own tessellation (volume, area, centroid) is the best free test of a reader. Some files record wrong values, though: zeros, unconverted units, and garbage centroids all occur.

10 Tessellated geometry (AP242)

AP242 can carry triangle meshes, either instead of exact geometry or alongside it:

```
#400=COORDINATES_LIST('',8,((0.,0.,0.),(1.,0.,0.),...));       shared vertex list
#401=COMPLEX_TRIANGULATED_FACE('',#400,4,((0.,0.,1.)),$,
       (1,2,3,4),((1,2,3,4)),());                              normals, pnindex, strips, fans
#402=TESSELLATED_SOLID('',(#401,...),$);
#403=TESSELLATED_SHAPE_REPRESENTATION('',(#402),#34);
```

`TRIANGULATED_FACE` lists triangles explicitly. `COMPLEX_TRIANGULATED_FACE` uses triangle strips and fans. Indices are 1-based into the coordinates list, through the face's `pnindex` when present. A tessellated item may point to the exact face it approximates.

- **Strips and fans:** Part 42 orders a strip's triangles (1, 2, 3), (3, 2, 4), …, each turning the same way as the first, and a fan's (1, 2, 3), (1, 3, 4), …. A strip repeats a corner where it needs to turn, which makes a degenerate triangle to skip. OCCT 8.1 builds both the other way round, so its meshes of tessellated solids come out inside out; the files' own normals agree with Part 42's order.
- **Normals:** none, one for the whole face, or one per point.
- **Links:** a face's `geometric_link` points to the exact face it approximates, and a shell's or solid's `topological_link` to its B-rep counterpart. A part may carry both its B-rep and a tessellation of it, in representations linked as in section 6.
- **Other items:** `TESSELLATED_SHELL` (closed or open according to its link), `TRIANGULATED_SURFACE_SET`, `TESSELLATED_CURVE_SET` and `TESSELLATED_EDGE` for lines, `TESSELLATED_POINT_SET`, and `REPOSITIONED_TESSELLATED_ITEM`s in a set, each with a placement of its own.
- **Graphical PMI:** most tessellated items in real files aren't part geometry at all but presentation: the polylines and triangulated text of PMI annotations, in `TESSELLATED_GEOMETRIC_SET`s within annotation or draughting representations. Semantic PMI, by contrast, is tolerances and datums attached to shape aspects. The CAx-IF practices on PMI and on polyline presentation describe both.

11 What real files look like

A survey of several hundred real files (CAx-IF test files, NIST models, a large open-source CAD test set) shows the problems a robust reader must absorb. None of them should make a reader reject a file:

- **Entities from other schemas or editions.** AP203 files often carry AP214 presentation entities, AP214 files AP242 tessellation, and old AP214 edition 1 entities appear in edition 3 files. Falling back to the AP242 schema for unknown entity names resolves most of these.
- **Attribute-count mismatches** from other editions or broken writers. Keep the record positional and drop the names.
- **Derived-attribute disagreements.** AP214 derives `NAMED_UNIT.dimensions` for conversion-based units, yet writers give it a value. Other schemas don't derive it, yet writers write *.
- **SELECT values written without their type,** such as a trim parameter `(0.)` instead of `(PARAMETER_VALUE(0.))`, or doubly wrapped values (`BOX_HEIGHT(POSITIVE_RATIO_MEASURE(3.))`).
- **Kinds mismatched with the schema:** reals where it says INTEGER, integers written without a decimal point where it says REAL, `$` in mandatory attributes (very common for names).
- **Complex instances with their partial entities out of alphabetical order,** which Part 21 requires.
- **Lexical problems:**

  - unescaped apostrophes inside strings (`'Bob's part'`): an apostrophe that isn't followed by `,` or `)` is most likely part of the string
  - malformed escapes: `\X2\` runs that aren't a multiple of four hex digits, runs never closed by `\X0\`, a stray `\X0\`, and the non-standard `\N\` and `\T\`; an escape a reader doesn't know is best kept as written, backslash included
  - an unbalanced quote that swallows the rest of the file; a reader can resynchronize at the next line that starts with `#<digits>=`
  - empty parameters (`EDGE_CURVE('',#1,#2,#3, )`)
  - references to ids that don't exist
  - duplicate ids
  - references too large for 32 bits
  - non-ASCII bytes in strings (usually UTF-8, sometimes ISO 8859-1)
- **Context problems:** missing length units (millimetres is the usual assumption), unusual degree factors, colours in 0 to 255.
- **Writer-specific assembly conventions** (section 7).
- **Placements among a representation's items** that no transformation uses, such as a `'MCS2'` machining frame. These are coordinate systems that go with the geometry, not transforms of it. At least one widely used reader moves the geometry by them anyway.
- **Many unconnected root products** in one file, and products with no shape (material definitions, analysis models).
- **Names that aren't names:** representations named `NONE` or after their own id (`#609`), where the product's name is the better label. But `SOLID` and `SHELL` are real names in some files.
- **Representations with nothing to mesh:** only curves and points (wireframes, and "geometrically bounded surface" representations that hold only curves), 2D drawings, only annotations (`TEXT_LITERAL`, callouts), or broken ones. A bare `LINE` in a geometric set is unbounded and can't be drawn.
- **Missing p-curves.** Most files give an edge only its 3D curve, with no curve in each face's parameter space: in the survey, about 94 % of faces, in nearly every AP242 and AP203 file. A reader that needs p-curves to trim and mesh faces must compute them by projecting the edges onto the surfaces (OCCT leaves this to its shape healing).
- **Orientation.** A closed shell facing inward with nothing enclosing it is a surface model whose normals follow its winding, best turned outward. A real void (`BREP_WITH_VOIDS`) is a shell inside exactly one other, with negative volume. Some old exports reverse their voids twice.
- **Tessellated geometry is mostly PMI** (section 10): part geometry carried only as triangles is rare.

12 Complete examples

The files below are complete and valid AP242 (the third shown without the parts it shares with the second). Their comments (`/* ... */`, allowed wherever whitespace is) say what each part of the file does. They were checked by reading them with OCCT (through the open-source `occt-import-js`), which gave the expected volumes (to within the tessellation's error), colours and placements.

12.1 A single part: a cylinder with a red top face

A cylinder is the smallest interesting B-rep: three faces, three edges, two vertices. The side face is bounded by the bottom circle, the **seam** line (the straight edge where the cylindrical surface closes on itself, used twice, once in each direction), and the top circle backwards. Each planar end face is bounded by one circle. The faces are oriented so that their normals point out of the solid. The bottom plane's placement faces $-z$, so `same_sense` can be `.T.` for every face. The whole solid is styled grey, and an `OVER_RIDING_STYLED_ITEM` makes the top face red.

```
ISO-10303-21;
HEADER;
/* the header: what the file is, who wrote it, and which schema its data follows */
FILE_DESCRIPTION(('a cylinder with a red top face'),'2;1');      /* '2;1': implementation level */
FILE_NAME('cylinder.stp','2026-10-17T12:00:00',('author'),('organization'),
  'hand written','tutorial','');                                /* preprocessor, originating system, authorization */
FILE_SCHEMA(('AP242_MANAGED_MODEL_BASED_3D_ENGINEERING_MIM_LF { 1 0 10303 442 1 1 4 }'));
ENDSEC;
```

```
DATA;
/* product structure: the part, its version and definition, and the definition's shape */
#1=APPLICATION_CONTEXT('managed model based 3d engineering');
#2=APPLICATION_PROTOCOL_DEFINITION('international standard',
  'ap242_managed_model_based_3d_engineering',2014,#1);          /* not referenced: identifies the AP */
#3=PRODUCT_CONTEXT('',#1,'mechanical');
#4=PRODUCT_DEFINITION_CONTEXT('part definition',#1,'design');    /* 'design': the life-cycle stage */
#5=PRODUCT('CYL-1','Cylinder','',(#3));                          /* id (the part number), name, description */
#6=PRODUCT_DEFINITION_FORMATION('A','',#5);                      /* version A of the product */
#7=PRODUCT_DEFINITION('design','',#6,#4);                        /* what assemblies and shapes attach to */
#8=PRODUCT_DEFINITION_SHAPE('','',#7);                           /* the definition's shape, as a property */
#9=SHAPE_DEFINITION_REPRESENTATION(#8,#10);                      /* ... represented by #10 */
#10=ADVANCED_BREP_SHAPE_REPRESENTATION('cylinder',(#20,#30),#15);  /* items: origin placement, solid */
/* units and context: millimetres, radians, 0.001 mm uncertainty */
#11=(LENGTH_UNIT()NAMED_UNIT(*)SI_UNIT(.MILLI.,.METRE.));         /* complex instance; * = derived dimensions */
#12=(NAMED_UNIT(*)PLANE_ANGLE_UNIT()SI_UNIT($,.RADIAN.));          /* $: no SI prefix */
#13=(NAMED_UNIT(*)SI_UNIT($,.STERADIAN.)SOLID_ANGLE_UNIT());
#14=UNCERTAINTY_MEASURE_WITH_UNIT(LENGTH_MEASURE(0.001),#11,'distance_accuracy_value','');
#15=(GEOMETRIC_REPRESENTATION_CONTEXT(3)GLOBAL_UNCERTAINTY_ASSIGNED_CONTEXT((#14))
     GLOBAL_UNIT_ASSIGNED_CONTEXT((#11,#12,#13))REPRESENTATION_CONTEXT('',''));  /* 3D, these units */
/* the part's origin: a right-handed frame, z axis then x axis */
#16=CARTESIAN_POINT('',(0.,0.,0.));
#17=DIRECTION('',(0.,0.,1.));
#18=DIRECTION('',(1.,0.,0.));
#20=AXIS2_PLACEMENT_3D('origin',#16,#17,#18);                    /* location, axis (z), ref_direction (x) */
/* the solid: radius 10, height 20, axis along z */
#30=MANIFOLD_SOLID_BREP('cylinder',#31);
#31=CLOSED_SHELL('',(#40,#50,#60));                              /* side, bottom, top */
/* geometry: the two circles, the seam line and the three surfaces */
#21=CARTESIAN_POINT('',(0.,0.,20.));
#22=AXIS2_PLACEMENT_3D('',#21,#17,#18);                          /* the top plane and top circle, at z = 20 */
#23=DIRECTION('',(0.,0.,-1.));
#24=AXIS2_PLACEMENT_3D('',#16,#23,#18);                          /* the bottom plane, facing -z */
#25=CIRCLE('',#20,10.);                                          /* bottom circle, about the origin frame's z */
#26=CIRCLE('',#22,10.);                                          /* top circle */
#27=CARTESIAN_POINT('',(10.,0.,0.));
#28=CARTESIAN_POINT('',(10.,0.,20.));
#29=LINE('',#27,#19);                                            /* the seam line: through #27, along #19 */
#19=VECTOR('',#17,1.);                                           /* defined after use: order doesn't matter */
#32=CYLINDRICAL_SURFACE('',#20,10.);                             /* its normal points away from the axis */
#33=PLANE('',#24);
#34=PLANE('',#22);
/* topology: two vertices (where the seam meets the circles) and three edges */
#35=VERTEX_POINT('',#27);
#36=VERTEX_POINT('',#28);
#37=EDGE_CURVE('bottom',#35,#35,#25,.T.);                        /* a closed edge: starts and ends at #35 */
#38=EDGE_CURVE('top',#36,#36,#26,.T.);
#39=EDGE_CURVE('seam',#35,#36,#29,.T.);                          /* where the cylindrical surface closes */
/* the side: bottom circle, seam up, top circle backwards, seam down */
#40=ADVANCED_FACE('side',(#41),#32,.T.);                         /* .T.: face normal = surface normal */
#41=FACE_OUTER_BOUND('',#42,.T.);
#42=EDGE_LOOP('',(#43,#44,#45,#46));
#43=ORIENTED_EDGE('',*,*,#37,.T.);                               /* * *: derived start and end vertices */
#44=ORIENTED_EDGE('',*,*,#39,.T.);
#45=ORIENTED_EDGE('',*,*,#38,.F.);                               /* .F.: the edge used backwards */
#46=ORIENTED_EDGE('',*,*,#39,.F.);                               /* the seam again, the other way */
/* the bottom, on a plane facing -z */
#50=ADVANCED_FACE('bottom',(#51),#33,.T.);
#51=FACE_OUTER_BOUND('',#52,.T.);
#52=EDGE_LOOP('',(#53));
#53=ORIENTED_EDGE('',*,*,#37,.F.);                               /* anticlockwise seen from below */
/* the top, on a plane facing +z */
#60=ADVANCED_FACE('top',(#61),#34,.T.);
#61=FACE_OUTER_BOUND('',#62,.T.);
#62=EDGE_LOOP('',(#63));
#63=ORIENTED_EDGE('',*,*,#38,.T.);                               /* anticlockwise seen from above */
/* colours: the whole solid grey */
#70=COLOUR_RGB('grey',0.6,0.6,0.65);                             /* sRGB, components in [0, 1] */
#71=FILL_AREA_STYLE_COLOUR('',#70);
```

```
#72=FILL_AREA_STYLE('',(#71));
#73=SURFACE_STYLE_FILL_AREA(#72);
#74=SURFACE_SIDE_STYLE('',(#73));
#75=SURFACE_STYLE_USAGE(.BOTH.,#74);                              /* both sides of the surfaces */
#76=PRESENTATION_STYLE_ASSIGNMENT((#75));
#77=STYLED_ITEM('',(#76),#30);                                    /* applies the style to the solid */
/* ... and the top face red, overriding the solid's style there */
#80=DRAUGHTING_PRE_DEFINED_COLOUR('red');                         /* one of eight named colours */
#81=FILL_AREA_STYLE_COLOUR('',#80);
#82=FILL_AREA_STYLE('',(#81));
#83=SURFACE_STYLE_FILL_AREA(#82);
#84=SURFACE_SIDE_STYLE('',(#83));
#85=SURFACE_STYLE_USAGE(.BOTH.,#84);
#86=PRESENTATION_STYLE_ASSIGNMENT((#85));
#87=OVER_RIDING_STYLED_ITEM('',(#86),#60,#77);                    /* item: top face; overrides #77 */
#90=MECHANICAL_DESIGN_GEOMETRIC_PRESENTATION_REPRESENTATION('',(#77,#87),#15);  /* collects the styles */
ENDSEC;
END-ISO-10303-21;
```

Things to notice:

- **Instance order is free.** `#19` is used before it is defined, and the geometry is listed between the solid and its faces.
- **Placements do double duty.** `#20` is both the part's origin placement and the placement of the bottom circle and of the cylindrical surface.
- **Not every entity is referenced.** Nothing refers to `#2`, `#9` or `#90`. Readers find them by type: the product structure through `PRODUCT_DEFINITION`, the styles through `STYLED_ITEM`.

12.2 An assembly: a pin through a hole in a disc

Two parts placed by an assembly:

- **The disc:** radius 20, height 5, with a through hole of radius 5.1. It adds two things the cylinder lacks:
  - **inner bounds:** the top and bottom faces each have the outer circle as `FACE_OUTER_BOUND` and the hole's circle as a `FACE_BOUND`
  - **a face whose normal runs against its surface's:** the hole's wall points at the axis, into the hole, while a `CYLINDRICAL_SURFACE`'s normal points away from it. So the wall has `same_sense .F.`, and its loop runs the other way round.
- **The pin:** a cylinder like the one above, radius 5 and height 30.

The assembly's representation holds only placements, one per component. Each component is placed by a NAUO, a CDSR and an `ITEM_DEFINED_TRANSFORMATION`, which takes the component's origin placement (`#206`, `#306`) onto the assembly's placement for it (`#107`, `#108`). The pin's placement is 12.5 mm down, so the pin spans $z = -12.5$ to 17.5 and passes through the hole with 0.1 mm of clearance. As in real assemblies, the solids don't overlap. All the representations share one geometric context, `#15`. That is allowed and common, though writers often give each part its own.

```
ISO-10303-21;
HEADER;
/* the header: what the file is, who wrote it, and which schema its data follows */
FILE_DESCRIPTION(('an assembly of a pin through a hole in a disc'),'2;1');
FILE_NAME('assembly.stp','2026-10-17T12:00:00',('author'),('organization'),'hand written','tutorial','');
FILE_SCHEMA(('AP242_MANAGED_MODEL_BASED_3D_ENGINEERING_MIM_LF { 1 0 10303 442 1 1 4 }'));
ENDSEC;
DATA;
/* shared by everything below: application context, units, one geometric context, common points and directions */
#1=APPLICATION_CONTEXT('managed model based 3d engineering');
#2=APPLICATION_PROTOCOL_DEFINITION('international standard','ap242_managed_model_based_3d_engineering',2014,#1);
#3=PRODUCT_CONTEXT('',#1,'mechanical');
#4=PRODUCT_DEFINITION_CONTEXT('part definition',#1,'design');
#11=(LENGTH_UNIT()NAMED_UNIT(*)SI_UNIT(.MILLI.,.METRE.));
#12=(NAMED_UNIT(*)PLANE_ANGLE_UNIT()SI_UNIT($,.RADIAN.));
#13=(NAMED_UNIT(*)SI_UNIT($,.STERADIAN.)SOLID_ANGLE_UNIT());
#14=UNCERTAINTY_MEASURE_WITH_UNIT(LENGTH_MEASURE(0.001),#11,'distance_accuracy_value','');
```

```
#15=(GEOMETRIC_REPRESENTATION_CONTEXT(3)GLOBAL_UNCERTAINTY_ASSIGNED_CONTEXT((#14))
     GLOBAL_UNIT_ASSIGNED_CONTEXT((#11,#12,#13))REPRESENTATION_CONTEXT('',''));  /* used by every representation */
#16=CARTESIAN_POINT('',(0.,0.,0.));
#17=DIRECTION('',(0.,0.,1.));
#18=DIRECTION('',(1.,0.,0.));
#19=DIRECTION('',(0.,0.,-1.));
#20=VECTOR('',#17,1.);                                          /* +z, for the seam lines */
/* the assembly: a product like any other, whose representation holds only placements, one per component */
#100=PRODUCT('ASM-1','Pinned disc','',(#3));
#101=PRODUCT_DEFINITION_FORMATION('A','',#100);
#102=PRODUCT_DEFINITION('design','',#101,#4);
#103=PRODUCT_DEFINITION_SHAPE('','',#102);
#104=SHAPE_DEFINITION_REPRESENTATION(#103,#105);
#105=SHAPE_REPRESENTATION('assembly',(#106,#107,#108),#15);
#106=AXIS2_PLACEMENT_3D('origin',#16,#17,#18);
#107=AXIS2_PLACEMENT_3D('disc placement',#16,#17,#18);          /* the disc goes at the origin */
#109=CARTESIAN_POINT('',(0.,0.,-12.5));
#108=AXIS2_PLACEMENT_3D('pin placement',#109,#17,#18);          /* the pin goes 12.5 mm down */
/* part Disc: radius 20, height 5, with a through hole of radius 5.1 for the pin */
#200=PRODUCT('DISC-1','Disc','',(#3));
#201=PRODUCT_DEFINITION_FORMATION('A','',#200);
#202=PRODUCT_DEFINITION('design','',#201,#4);
#203=PRODUCT_DEFINITION_SHAPE('','',#202);
#204=SHAPE_DEFINITION_REPRESENTATION(#203,#205);
#205=ADVANCED_BREP_SHAPE_REPRESENTATION('Disc',(#206,#210),#15);
#206=AXIS2_PLACEMENT_3D('origin',#16,#17,#18);                  /* the disc's own origin: what gets placed */
#210=MANIFOLD_SOLID_BREP('Disc',#211);
#211=CLOSED_SHELL('',(#240,#244,#250,#260));                    /* outer side, hole, bottom, top */
/* disc geometry: four circles, two seam lines, two cylinders, two planes */
#221=CARTESIAN_POINT('',(0.,0.,5.));
#222=AXIS2_PLACEMENT_3D('',#221,#17,#18);                       /* the top plane and circles, at z = 5 */
#224=AXIS2_PLACEMENT_3D('',#16,#19,#18);                        /* the bottom plane, facing -z */
#225=CIRCLE('',#206,20.);                                       /* outer, bottom */
#226=CIRCLE('',#222,20.);                                       /* outer, top */
#227=CIRCLE('',#206,5.1);                                       /* hole, bottom */
#228=CIRCLE('',#222,5.1);                                       /* hole, top */
#229=CARTESIAN_POINT('',(20.,0.,0.));
#230=CARTESIAN_POINT('',(20.,0.,5.));
#231=CARTESIAN_POINT('',(5.1,0.,0.));
#232=CARTESIAN_POINT('',(5.1,0.,5.));
#233=LINE('',#229,#20);                                         /* outer seam */
#234=LINE('',#231,#20);                                         /* hole seam */
#235=CYLINDRICAL_SURFACE('',#206,20.);
#236=CYLINDRICAL_SURFACE('',#206,5.1);                          /* normal points away from the axis... */
#237=PLANE('',#224);
#238=PLANE('',#222);
/* disc topology: four vertices, six edges */
#212=VERTEX_POINT('',#229);
#213=VERTEX_POINT('',#230);
#214=VERTEX_POINT('',#231);
#215=VERTEX_POINT('',#232);
#216=EDGE_CURVE('outer bottom',#212,#212,#225,.T.);
#217=EDGE_CURVE('outer top',#213,#213,#226,.T.);
#218=EDGE_CURVE('hole bottom',#214,#214,#227,.T.);
#219=EDGE_CURVE('hole top',#215,#215,#228,.T.);
#220=EDGE_CURVE('outer seam',#212,#213,#233,.T.);
#223=EDGE_CURVE('hole seam',#214,#215,#234,.T.);
/* the outer side, as for the cylinder */
#240=ADVANCED_FACE('outer side',(#241),#235,.T.);
#241=FACE_OUTER_BOUND('',#242,.T.);
#242=EDGE_LOOP('',(#270,#271,#272,#273));
#270=ORIENTED_EDGE('',*,*,#216,.T.);
#271=ORIENTED_EDGE('',*,*,#220,.T.);
#272=ORIENTED_EDGE('',*,*,#217,.F.);
#273=ORIENTED_EDGE('',*,*,#220,.F.);
/* the hole's wall */
#244=ADVANCED_FACE('hole',(#245),#236,.F.);                     /* ...but this face's points at it: .F. */
#245=FACE_OUTER_BOUND('',#246,.T.);
#246=EDGE_LOOP('',(#274,#275,#276,#277));                       /* so its loop runs the other way round */
```

```
#274=ORIENTED_EDGE('',*,*,#223,.T.);
#275=ORIENTED_EDGE('',*,*,#219,.T.);
#276=ORIENTED_EDGE('',*,*,#223,.F.);
#277=ORIENTED_EDGE('',*,*,#218,.F.);
/* the bottom, facing -z: the outer circle, and the hole as an inner bound */
#250=ADVANCED_FACE('bottom',(#251,#253),#237,.T.);
#251=FACE_OUTER_BOUND('',#252,.T.);
#252=EDGE_LOOP('',(#278));
#278=ORIENTED_EDGE('',*,*,#216,.F.);
#253=FACE_BOUND('',#254,.T.);                                   /* an inner bound: the hole */
#254=EDGE_LOOP('',(#279));
#279=ORIENTED_EDGE('',*,*,#218,.T.);                            /* inner loops turn opposite to outer ones */
/* the top, facing +z */
#260=ADVANCED_FACE('top',(#261,#263),#238,.T.);
#261=FACE_OUTER_BOUND('',#262,.T.);
#262=EDGE_LOOP('',(#280));
#280=ORIENTED_EDGE('',*,*,#217,.T.);
#263=FACE_BOUND('',#264,.T.);
#264=EDGE_LOOP('',(#281));
#281=ORIENTED_EDGE('',*,*,#219,.F.);
/* disc colour: grey */
#290=COLOUR_RGB('grey',0.6,0.6,0.65);
#291=FILL_AREA_STYLE_COLOUR('',#290);
#292=FILL_AREA_STYLE('',(#291));
#293=SURFACE_STYLE_FILL_AREA(#292);
#294=SURFACE_SIDE_STYLE('',(#293));
#295=SURFACE_STYLE_USAGE(.BOTH.,#294);
#296=PRESENTATION_STYLE_ASSIGNMENT((#295));
#297=STYLED_ITEM('',(#296),#210);
#298=MECHANICAL_DESIGN_GEOMETRIC_PRESENTATION_REPRESENTATION('',(#297),#15);
/* part Pin: a cylinder of radius 5 and height 30, built exactly like the single-part example */
#300=PRODUCT('PIN-1','Pin','',(#3));
#301=PRODUCT_DEFINITION_FORMATION('A','',#300);
#302=PRODUCT_DEFINITION('design','',#301,#4);
#303=PRODUCT_DEFINITION_SHAPE('','',#302);
#304=SHAPE_DEFINITION_REPRESENTATION(#303,#305);
#305=ADVANCED_BREP_SHAPE_REPRESENTATION('Pin',(#306,#310),#15);
#306=AXIS2_PLACEMENT_3D('origin',#16,#17,#18);                  /* the pin's own origin */
#310=MANIFOLD_SOLID_BREP('Pin',#311);
#311=CLOSED_SHELL('',(#340,#350,#360));
#321=CARTESIAN_POINT('',(0.,0.,30.));
#322=AXIS2_PLACEMENT_3D('',#321,#17,#18);
#324=AXIS2_PLACEMENT_3D('',#16,#19,#18);
#325=CIRCLE('',#306,5.);
#326=CIRCLE('',#322,5.);
#327=CARTESIAN_POINT('',(5.,0.,0.));
#328=CARTESIAN_POINT('',(5.,0.,30.));
#329=LINE('',#327,#20);
#332=CYLINDRICAL_SURFACE('',#306,5.);
#333=PLANE('',#324);
#334=PLANE('',#322);
#335=VERTEX_POINT('',#327);
#336=VERTEX_POINT('',#328);
#337=EDGE_CURVE('',#335,#335,#325,.T.);
#338=EDGE_CURVE('',#336,#336,#326,.T.);
#339=EDGE_CURVE('',#335,#336,#329,.T.);
#340=ADVANCED_FACE('',(#341),#332,.T.);
#341=FACE_OUTER_BOUND('',#342,.T.);
#342=EDGE_LOOP('',(#343,#344,#345,#346));
#343=ORIENTED_EDGE('',*,*,#337,.T.);
#344=ORIENTED_EDGE('',*,*,#339,.T.);
#345=ORIENTED_EDGE('',*,*,#338,.F.);
#346=ORIENTED_EDGE('',*,*,#339,.F.);
#350=ADVANCED_FACE('',(#351),#333,.T.);
#351=FACE_OUTER_BOUND('',#352,.T.);
#352=EDGE_LOOP('',(#353));
#353=ORIENTED_EDGE('',*,*,#337,.F.);
#360=ADVANCED_FACE('',(#361),#334,.T.);
#361=FACE_OUTER_BOUND('',#362,.T.);
```

```
#362=EDGE_LOOP('',(#363));
#363=ORIENTED_EDGE('',*,*,#338,.T.);
/* pin colour: red */
#370=DRAUGHTING_PRE_DEFINED_COLOUR('red');
#371=FILL_AREA_STYLE_COLOUR('',#370);
#372=FILL_AREA_STYLE('',(#371));
#373=SURFACE_STYLE_FILL_AREA(#372);
#374=SURFACE_SIDE_STYLE('',(#373));
#375=SURFACE_STYLE_USAGE(.BOTH.,#374);
#376=PRESENTATION_STYLE_ASSIGNMENT((#375));
#377=STYLED_ITEM('',(#376),#310);
#378=MECHANICAL_DESIGN_GEOMETRIC_PRESENTATION_REPRESENTATION('',(#377),#15);
/* the disc in the assembly: the occurrence, its shape, and the placement */
#400=NEXT_ASSEMBLY_USAGE_OCCURRENCE('1','disc','',#102,#202,$);  /* parent ASM-1, child DISC-1 */
#401=PRODUCT_DEFINITION_SHAPE('','',#400);                       /* the shape of this occurrence */
#402=ITEM_DEFINED_TRANSFORMATION('','',#206,#107);               /* the disc's origin onto its placement */
#403=(REPRESENTATION_RELATIONSHIP('','',#205,#105)                /* rep_1 the component, rep_2 the assembly */
      REPRESENTATION_RELATIONSHIP_WITH_TRANSFORMATION(#402)
      SHAPE_REPRESENTATION_RELATIONSHIP());
#404=CONTEXT_DEPENDENT_SHAPE_REPRESENTATION(#403,#401);          /* ties the placement to the occurrence */
/* the pin in the assembly, 12.5 mm down so it passes through the hole */
#410=NEXT_ASSEMBLY_USAGE_OCCURRENCE('2','pin','',#102,#302,$);
#411=PRODUCT_DEFINITION_SHAPE('','',#410);
#412=ITEM_DEFINED_TRANSFORMATION('','',#306,#108);
#413=(REPRESENTATION_RELATIONSHIP('','',#305,#105)
      REPRESENTATION_RELATIONSHIP_WITH_TRANSFORMATION(#412)
      SHAPE_REPRESENTATION_RELATIONSHIP());
#414=CONTEXT_DEPENDENT_SHAPE_REPRESENTATION(#413,#411);
ENDSEC;
END-ISO-10303-21;
```

Things to notice:

- **To use the same part twice,** add a second NAUO with its own CDSR to the same `PRODUCT_DEFINITION` (`#302`, say), rather than copying the part.
- **The relationships follow the standard order:** `rep_1` is the component's representation, `rep_2` the assembly's, and `transform_item_1` lies in `rep_1`. Section 7 lists the variations real files use.
- **Assembly-level colours:** to colour the pin differently in this assembly only, a `CONTEXT_DEPENDENT_OVER_RIDING_STYLED_ITEM` would list `#413` in its `style_context`.

12.3 The same assembly, placed by mapped items (the explicit method)

The pinned disc again, this time placed the explicit way (section 7). The assembly's representation holds a `MAPPED_ITEM` for each component, and each occurrence's own `SHAPE_DEFINITION_REPRESENTATION` holds its mapped item. There are no CDSRs and no `ITEM_DEFINED_TRANSFORMATION`s. The two parts are exactly as in the previous example, so they are left out here: the full file is that example with the assembly and occurrence sections replaced by these.

```
ISO-10303-21;
HEADER;
/* the header: what the file is, who wrote it, and which schema its data follows */
FILE_DESCRIPTION(('the pinned disc, placed by mapped items (the explicit method)'),'2;1');
FILE_NAME('assembly_mapped.stp','2026-10-17T12:00:00',('author'),('organization'),'hand written','tutorial','');
FILE_SCHEMA(('AP242_MANAGED_MODEL_BASED_3D_ENGINEERING_MIM_LF { 1 0 10303 442 1 1 4 }'));
ENDSEC;
DATA;
/* shared by everything below: application context, units, one geometric context, common points and directions */
#1=APPLICATION_CONTEXT('managed model based 3d engineering');
#2=APPLICATION_PROTOCOL_DEFINITION('international standard','ap242_managed_model_based_3d_engineering',2014,#1);
#3=PRODUCT_CONTEXT('',#1,'mechanical');
#4=PRODUCT_DEFINITION_CONTEXT('part definition',#1,'design');
#11=(LENGTH_UNIT()NAMED_UNIT(*)SI_UNIT(.MILLI.,.METRE.));
#12=(NAMED_UNIT(*)PLANE_ANGLE_UNIT()SI_UNIT($,.RADIAN.));
#13=(NAMED_UNIT(*)SI_UNIT($,.STERADIAN.)SOLID_ANGLE_UNIT());
#14=UNCERTAINTY_MEASURE_WITH_UNIT(LENGTH_MEASURE(0.001),#11,'distance_accuracy_value','');
#15=(GEOMETRIC_REPRESENTATION_CONTEXT(3)GLOBAL_UNCERTAINTY_ASSIGNED_CONTEXT((#14))
```

```
     GLOBAL_UNIT_ASSIGNED_CONTEXT((#11,#12,#13))REPRESENTATION_CONTEXT('',''));  /* used by every representation */
#16=CARTESIAN_POINT('',(0.,0.,0.));
#17=DIRECTION('',(0.,0.,1.));
#18=DIRECTION('',(1.,0.,0.));
#19=DIRECTION('',(0.,0.,-1.));
#20=VECTOR('',#17,1.);                                          /* +z, for the seam lines */
/* the assembly: its representation holds one MAPPED_ITEM per component, so its contents are the placed components */
#100=PRODUCT('ASM-1','Pinned disc','',(#3));
#101=PRODUCT_DEFINITION_FORMATION('A','',#100);
#102=PRODUCT_DEFINITION('design','',#101,#4);
#103=PRODUCT_DEFINITION_SHAPE('','',#102);
#104=SHAPE_DEFINITION_REPRESENTATION(#103,#105);
#105=SHAPE_REPRESENTATION('assembly',(#106,#120,#121),#15);
#106=AXIS2_PLACEMENT_3D('origin',#16,#17,#18);
#107=AXIS2_PLACEMENT_3D('disc placement',#16,#17,#18);          /* the disc goes at the origin */
#109=CARTESIAN_POINT('',(0.,0.,-12.5));
#108=AXIS2_PLACEMENT_3D('pin placement',#109,#17,#18);          /* the pin goes 12.5 mm down */
#110=REPRESENTATION_MAP(#206,#205);                             /* the disc's representation, mapped from its origin
*/
#120=MAPPED_ITEM('disc',#110,#107);                             /* ... onto the disc placement */
#111=REPRESENTATION_MAP(#306,#305);                             /* the pin's representation, from its origin */
#121=MAPPED_ITEM('pin',#111,#108);                              /* ... onto the pin placement */
/* ... the two parts, #200 to #378, exactly as in the previous example ... */
/* the disc in the assembly: the occurrence, and its shape, represented by the mapped item that places the disc */
#400=NEXT_ASSEMBLY_USAGE_OCCURRENCE('1','disc','',#102,#202,$);  /* parent ASM-1, child DISC-1 */
#401=PRODUCT_DEFINITION_SHAPE('','',#400);                      /* the shape of this occurrence... */
#402=SHAPE_DEFINITION_REPRESENTATION(#401,#403);                /* ...is represented by #403 */
#403=SHAPE_REPRESENTATION('disc occurrence',(#120),#15);        /* which holds just the mapped item */
/* the pin in the assembly */
#410=NEXT_ASSEMBLY_USAGE_OCCURRENCE('2','pin','',#102,#302,$);
#411=PRODUCT_DEFINITION_SHAPE('','',#410);
#412=SHAPE_DEFINITION_REPRESENTATION(#411,#413);
#413=SHAPE_REPRESENTATION('pin occurrence',(#121),#15);
ENDSEC;
END-ISO-10303-21;
```

Things to notice:

- **Each mapped item appears twice:** in the assembly's representation (`#105`), so the assembly's geometry is the placed components, and in its occurrence's representation (`#403`, `#413`), which ties it to the NAUO.
- **The placements are the same ones the implicit version used** (`#107`, `#108`). Only the mechanism differs: `REPRESENTATION_MAP(#206,#205)` maps the disc from its origin, where the implicit version's `ITEM_DEFINED_TRANSFORMATION(#206,#107)` transformed it from there.

13 References

- **CAx-IF / MBx-IF Recommended Practices** (https://www.mbx-if.org/home/cax/recpractices/): how to use the APs for real data. Each document has worked Part 21 examples. The most useful for reading files:
  - the PDM Schema Usage Guide (product structure and assemblies)
  - Model Styling and Organization (colours, layers)
  - Geometric and Assembly Validation Properties
  - External References
  - 3D Tessellated Geometry
  - Representation and Presentation of PMI
  - User Defined Attributes
- **EXPRESS schemas:** the AP long forms are published openly. The open-source stepcode project (https://github.com/stepcode/stepcode) includes AP203, AP214 and AP242 schemas in its `data` directory.
- **Test files:**
  - NIST MBE PMI models (https://www.nist.gov/ctl/smart-connected-systems-division/smart-connected-manufacturing-systems-group/mbe-pmi-0): AP203 and AP242 files with PMI and validation properties
  - the CAx-IF test-round files that open-source projects collect

- **Open-source readers to compare against:**
  - Open CASCADE Technology (https://github.com/Open-Cascade-SAS/OCCT): STEP reading in `src/DataExchange/TKDESTEP`, with assembly, colour and layer handling in its XCAF layer
  - NIST's STEP File Analyzer and Viewer (https://github.com/usnistgov/SFA): checks files against the recommended practices
  - stepcode
- **ISO 10303-21** itself costs money from ISO, but the format is simple. Section 2 covers what readers meet in practice, and the edition 3 additions (anchors, references, several data sections, signatures) are rare in CAD exports.